

Manual MLOps Reflection: Challenges, Lessons, and Insights

Shlok Shetty , MM22B003

1. Most Difficult Part of Managing Data Versions Manually

The hardest part of manual management of versions was keeping a **mental and physical mapping of the data files, configurations, and resulting versions of the project**. In manual versioning, we had to keep track of:

- What raw data was used to create which version of the data,
- What processing was done in which version of the data,
- What model was used with which version of the data.

The hardest part of manual versioning was not actually creating versions, but remembering to respect the versioning discipline of considering each version of the data immutable once it was used, and documenting each change clearly in the manifest and the log files. The manual versioning process made me realize just how easy it is to lose track and get lost in a real project without adequate tooling or conventions.

2. Ensuring the Model in Production Matched the Evaluated Model

In order to ensure that the model used in the deployment was the same model used during training, versioning of the model was used. Therefore, each model was:

- Saved with a versioned filename, e.g., model_vX.pkl
- Linked with a metadata JSON file with the following information:

Training date, Version of the data, Hyperparameters, Evaluation metrics and Commit hash of the Git repository

The API did not guess which model to use. **Instead, it read the active version of the model from config.yaml, built the model path dynamically, and then used that model. This ensured that the model used in the deployment was linked to a specific version.** Deployment logs were also recorded in deployment_log.csv. This ensured that it was possible to audit which version of the model was being used at a specific point in time. This eliminated the possibility of confusion between models that may have had similar performance but different training histories.

3. Ensuring train.py Used the Correct Data Version

The key to making sure that train.py always used the correct data version was the single source of truth approach to configuration. The data version was specified in a single place: the config.yaml file, under the data.version field. All file path strings in train.py were generated programmatically using the data version (e.g., v2_train.csv, v2_test.csv). No hard-coded file names were used inside the training script. This means that:

Changing the data version required only modifying the YAML file,

Re-running train.py would then properly consume the correct data set,

And the possibility of mixing data versions was eliminated.

4. Top Three Features Desired in an Automated MLOps Tool

Based on this experience, here are the top three things that I would like to see in an automated MLOps tool:

- **Automated tracking and management of data and models:** The process of creating a manifest file made me understand that human management of lineage is very error-prone. An automated system would help keep a precise record of the data and model versions along with the transformations made to them.
- **Model registry:** The process of managing multiple versions of a model across training, evaluation, and deployment was where most human error happened. An automated model registry would help me easily determine which model is ready for staging, production etc..
- **Automated monitoring with log partitioning.** I had a very hard time keeping track large number of logs which were script generated. An automated system that partitions logs by model version, deployment instance, and time window would greatly help us monitor my models

5. Breakdown in Tracking the Active Model Serving Requests

The biggest snag was trying to figure out what model version was actually serving the API requests. Because the prediction log was append-only without any division by model version or deployment window, there was considerable bleed between the inference phases—smoke tests, Day-2 inference, debugging, etc. So, what we ended up seeing was:

- the same monitoring script slam-dancing with different metrics from run to run,
- the model's performance was non-deterministic at first glance,
- and whether the model was indeed performing worse or if this was just a side effect of the logging.

The final issue was not with the model at all, but with the lack of a well-defined boundary between inference sessions. This was a rather obvious example of how observability fails without good logging and deployment identification.

6. How an Automated Model Registry Would Have Simplified Phase B

An Automated Model Registry would have significantly smoothed Phase B by removing the necessity of hand-holding training, evaluation, and deployment together. This would have:

- eliminated the necessity of manually naming model artifacts,
- automatically associated metrics and metadata with models,
- provided a shared and unambiguous understanding of the “current production model,” and
- eliminated the necessity of adjusting configuration files to promote or roll back models.

Instead of manipulating YAMLs and restarting services, the deployment would have been thought of as a simple state transition (register → promote → serve). This would have reduced cognitive burden and prevented several near-misses of deploying the wrong model.